DevOps Project: End-to-End CI/CD Pipeline Using Jenkins, Docker, GitHub, and Docker Hub (Local Ubuntu Environment)

Project Overview

This project demonstrates a complete Continuous Integration and Continuous Deployment (CI/CD) pipeline using:

- GitHub as Source Code Repository
- Jenkins as CI/CD Tool
- Docker for Containerization
- Docker Hub as Container Registry
- Ubuntu Local Machine as Deployment Server

The goal is to automatically build, package, store, and deploy a Node.js application whenever code changes are pushed to GitHub.

Why Build This Project Locally Instead of AWS?

As a beginner, setting up the entire DevOps workflow locally has several advantages:

Cost Saving

- No AWS charges
- No EC2 costs
- No EBS costs
- No data transfer costs

Easy Troubleshooting

Since Jenkins, Docker, and the application are on the same machine:

- Easier debugging
- Easier log collection
- Faster testing

Learning Focus

The objective is to understand:

- CI/CD concepts
- Jenkins pipelines
- Docker image creation
- Docker Hub integration
- Automated deployments

without worrying about cloud networking, security groups, IAM roles, VPCs, or DNS configurations.

Industry Workflow Understanding

The same pipeline can later be migrated to:

- AWS EC2
- Azure VM
- Google Cloud VM
- Kubernetes Cluster

The concepts remain exactly the same.

Project Architecture

```
Developer
    |
    v
GitHub Repository
    |
    v
```

```
Jenkins Pipeline
    |
    +--> Build Docker Image
    |
    +--> Push Image to Docker Hub
    |
    +--> Pull Latest Image
    |
    +--> Deploy Container
    |
    v
Docker Container
    |
    v
Node.js Application
```

Project Requirements

Hardware Requirements

Minimum:

- 4 GB RAM
- 2 CPU Cores
- 20 GB Free Disk

Recommended:

- 8 GB RAM
- 4 CPU Cores

Software Requirements

Operating System

Ubuntu 22.04 LTS

Required Tools

- Java 17
- Jenkins
- Docker
- Git
- Node.js
- Docker Hub Account
- GitHub Account

CI/CD Workflow

```
Developer Pushes Code
            |
            v
          GitHub
            |
            v
          Jenkins
            |
            v
      Build Docker Image
            |
            v
      Push To Docker Hub
            |
            v
      Pull Latest Image
```

```
        |
        v
Deploy Container
        |
        v
Application Running
```

Step 1: Install Java

Official Site:

```
https://www.jenkins.io/doc/book/installing/
```

```
sudo apt update

sudo apt install openjdk-17-jdk -y

java -version
```

Step 2: Install Jenkins

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee /usr/share/keyrings/jenkins-keyr
```

```
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/ | suc
```

```
sudo apt update

sudo apt install jenkins -y
```

Start Jenkins:

```
sudo systemctl enable jenkins

sudo systemctl start jenkins
```

Verify:

```
sudo systemctl status jenkins
```

Get Admin Password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Access:

```
http://localhost:8080
```

Install Suggested Plugins.

Step 3: Install Docker

```
sudo apt install docker.io -y
```

Start Docker:

```
sudo systemctl enable docker

sudo systemctl start docker
```

Verify:

```
docker --version
```

Step 4: Give Jenkins Access To Docker

```
sudo usermod -aG docker jenkins
```

Restart Services:

```
sudo systemctl restart docker

sudo systemctl restart jenkins
```

Verify:

```
groups jenkins
```

Expected:

```
jenkins docker
```

Step 5: Install Git

```
sudo apt install git -y
```

Verify:

```
git --version
```

Step 6: Create Docker Hub Account

Create Docker Hub Account.

Create Repository:

```
nodejs-cicd
```

Step 7: Create Node.js Application

Create Project Directory:

```
mkdir nodejs-cicd

cd nodejs-cicd
```

Create app.js

```
const express = require('express');
```

```
const app = express();

app.get('/', (req, res) => {
    res.send('DevOps CI/CD Project Working');
});

app.listen(3000, () => {
    console.log('Running on port 3000');
});
```

Create package.json

```
{
  "name": "nodejs-cicd",
  "version": "1.0.0",
  "scripts": {
    "start": "node app.js",
    "test": "echo Test Passed"
  },
  "dependencies": {
    "express": "^5.1.0"
  }
}
```

Install Dependencies:

```
npm install
```

Step 8: Create Dockerfile

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm","start"]
```

Step 9: Push Code To GitHub

```
git init

git add .

git commit -m "Initial Commit"

git branch -M main

git remote add origin YOUR_REPOSITORY_URL

git push -u origin main
```

Step 10: Jenkins Plugins

Install:

- Pipeline
- Git
- Git Client
- Docker Pipeline
- Credentials Binding
- GitHub

Step 11: Add Docker Hub Credentials

Navigate:

```
Manage Jenkins
  -> Credentials
  -> System
  -> Global Credentials
```

Add:

```
Username : Docker Hub Username
Password : Docker Hub Password / Access Token
ID       : dockerhub
```

Step 12: Create Jenkins Pipeline Job

```
New Item
   |
Pipeline
   |
NodeJS-CICD
```

Step 13: Jenkinsfile

Create Jenkinsfile in Project Root.

```
pipeline {

    agent any

    environment {
        IMAGE_NAME = "YOUR_DOCKERHUB_USERNAME/nodejs-cicd"
        CONTAINER_NAME = "nodejs-app"
    }

    stages {

        stage('Checkout') {
            steps {
                git branch: 'main',
                url: 'YOUR_GITHUB_REPOSITORY_URL'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t ${IMAGE_NAME}:latest .'
            }
        }

        stage('Docker Login') {
            steps {
```

```
                withCredentials([
                    usernamePassword(
                        credentialsId: 'dockerhub',
                        usernameVariable: 'DOCKER_USER',
                        passwordVariable: 'DOCKER_PASS'
                    )
                ]) {
                    sh '''
                    echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin
                    '''
                }
            }
        }

        stage('Push Image') {
            steps {
                sh 'docker push ${IMAGE_NAME}:latest'
            }
        }

        stage('Deploy Container') {
            steps {
                sh '''
                docker stop ${CONTAINER_NAME} || true
                docker rm ${CONTAINER_NAME} || true

                docker pull ${IMAGE_NAME}:latest

                docker run -d \
                --name ${CONTAINER_NAME} \
                -p 3000:3000 \
                ${IMAGE_NAME}:latest
                '''
            }
        }
    }
}
```

Step 14: Execute Pipeline

Click:

```
Build Now
```

Pipeline Flow:

```
GitHub Checkout
      |
Docker Build
      |
Docker Login
      |
Docker Push
      |
Docker Pull
      |
Deploy Container
      |
Application Live
```

Verification

Check Running Container:

```
docker ps
```

Check Images:

```
docker images
```

Open Browser:

```
http://localhost:3000
```

Expected Output:

```
DevOps CI/CD Project Working
```

Key Learning Outcomes

After completing this project, you will understand:

- CI/CD Fundamentals
- Jenkins Pipeline Creation
- Docker Image Creation
- Docker Hub Integration
- Automated Deployment
- GitHub Integration
- Container Lifecycle Management
- Real-World DevOps Workflow

These enhancements will make the project closer to a production-grade DevOps platform.

Support the Channel ❤️

If you found this project helpful and learned something new, please:

👍 Like the video 💬 Leave a comment with your feedback or questions 🔁 Share it with your friends and fellow DevOps learners 🔔 Subscribe to the channel for more DevOps, Cloud, Docker, Kubernetes, Jenkins, and CI/CD tutorials

Your support helps us create more real-world projects and learning content for the community.

Thank you for watching and happy learning! 🚀

Presented by DevOps ZPlus Tutorials